

WebSocket_Monitoring_Guide

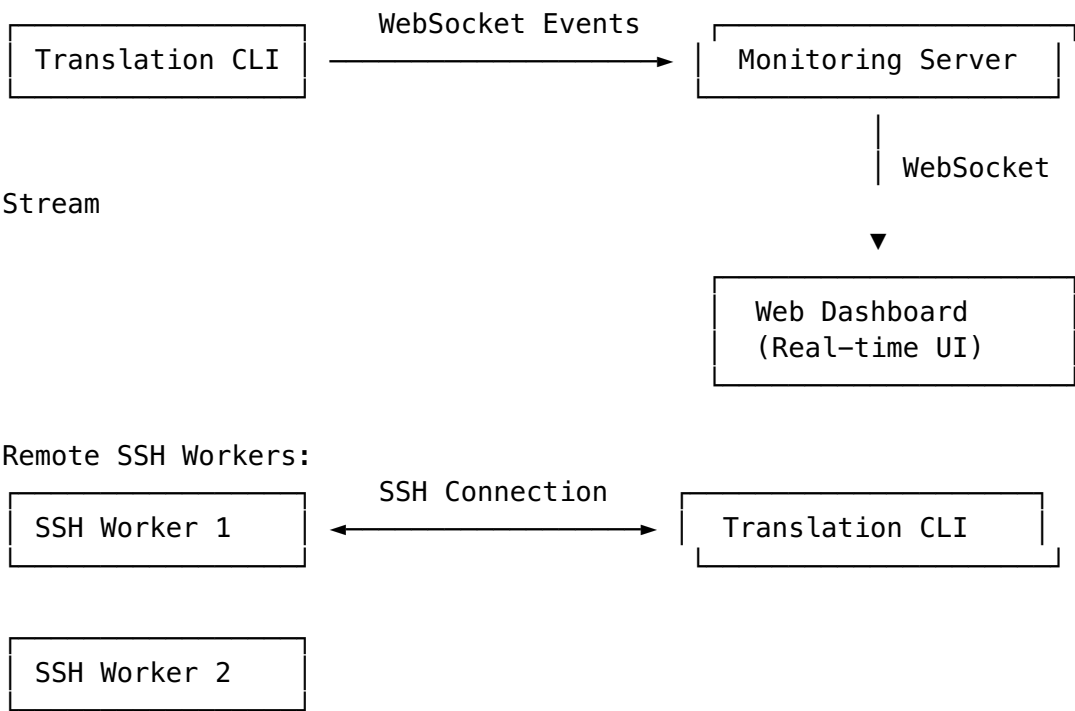
WebSocket Monitoring System Documentation

Overview

The Translation Monitor system provides real-time WebSocket-based monitoring for translation workflows, supporting both local LLM translation and remote SSH worker translation. The system consists of:

- **WebSocket Server:** Real-time event streaming server
- **Monitoring Dashboard:** Interactive web interface for tracking progress
- **Event System:** Comprehensive event emission and handling
- **SSH Worker Integration:** Remote worker management and monitoring

Architecture



Components

1. WebSocket Server (`cmd/monitor-server/main.go`)

Real-time event streaming server running on port 8090.

Features:

- WebSocket connection management
- Event routing to clients
- Session-based monitoring
- Multi-client support

Endpoints:

- `ws://localhost:8090/ws?session_id={id}&client_id={id}` - WebSocket connection
- `http://localhost:8090/monitor` - Web dashboard

Usage:

```
go run ./cmd/monitor-server
```

2. Monitoring Dashboard (`monitor.html` & `enhanced-monitor.html`)

Interactive web interface for monitoring translation progress.

Features:

- Real-time progress bars
- Event logging
- Session history
- Worker information display
- Progress charts

Access:

Basic monitor

open `http://localhost:8090/monitor`

Enhanced monitor with SSH worker support

open `enhanced-monitor.html`

3. Event System (`pkg/events/events.go`)

Event-driven architecture for translation monitoring.

Event Types:

- `translation_started` - Translation job initiated
- `translation_progress` - Progress update
- `translation_completed` - Translation finished

- translation_error - Error occurred
- step_completed - Step finished
- conversion_started - Content conversion started
- conversion_progress - Conversion progress
- conversion_completed - Conversion finished

Event Structure:

```
{
  "type": "translation_progress",
  "session_id": "session-123",
  "step": "translation",
  "message": "Translating line 5/10",
  "progress": 55.0,
  "current_item": "line_5",
  "total_items": 10,
  "timestamp": 1640995200,
  "data": {
    "worker_info": {
      "host": "localhost",
      "port": 8444,
      "type": "ssh-llamacpp"
    }
  }
}
```

4. SSH Worker Integration (pkg/sshworker/)

Remote worker management for distributed translation.

Features:

- SSH connection management
- Remote command execution
- Progress tracking
- Error handling
- Worker health monitoring

Configuration:

```
{
  "distributed": {
    "enabled": true,
    "workers": {
      "thinker-worker": {
        "name": "Local Llama.cpp Worker",
        "host": "localhost",
        "port": 8444,
        "user": "milosvasic",
        "password": "password",
        "max_capacity": 10,
        "enabled": true,
        "tags": ["gpu", "llamacpp"]
      }
    }
  }
}
```

```
    }  
  },  
  "ssh_timeout": 30,  
  "ssh_max_retries": 3  
}  
}
```

Quick Start Guide

1. Start the Monitoring Server

```
# Navigate to project root  
cd /Users/milosvasic/Projects/Translate  
  
# Start WebSocket monitoring server  
go run ./cmd/monitor-server
```

2. Open the Monitoring Dashboard

```
# Open web dashboard (in new terminal)  
open http://localhost:8090/monitor  
  
# Or open the enhanced dashboard  
open enhanced-monitor.html
```

3. Run Translation with Monitoring

Option A: Basic Demo with WebSocket Monitoring

```
# Run the basic translation demo  
go run demo-translation-with-monitoring-fixed.go
```

Option B: Real LLM Translation with Monitoring

```
# Set OpenAI API key (optional)  
export OPENAI_API_KEY=your_api_key_here  
  
# Run real LLM translation  
go run demo-real-llm-with-monitoring.go
```

Option C: SSH Worker Translation with Monitoring

```
# Configure SSH worker (optional)  
export SSH_WORKER_HOST=localhost  
export SSH_WORKER_USER=milosvasic  
export SSH_WORKER_PASSWORD=your_password  
  
# Run SSH worker translation  
go run demo-ssh-worker-with-monitoring.go
```

4. Monitor Progress

The dashboard will show:

- Real-time progress updates
- Current translation step
- Event logs
- Session history
- Worker information (for SSH mode)

Configuration

Environment Variables

```
# WebSocket Server
MONITOR_SERVER_PORT=8090

# SSH Workers
SSH_WORKER_HOST=localhost
SSH_WORKER_USER=milosvasic
SSH_WORKER_PASSWORD=password
SSH_WORKER_PORT=22
SSH_WORKER_REMOTE_DIR=/tmp/translate-ssh

# LLM Configuration
OPENAI_API_KEY=your_openai_key
ANTHROPIC_API_KEY=your_anthropic_key
DEEPSEEK_API_KEY=your_deepseek_key
```

Configuration Files

- config.json - Main application configuration
- internal/working/config.distributed.json - SSH worker configuration
- internal/working/config.*.json - Various LLM provider configurations

API Reference

WebSocket Messages

Client to Server

```
// Connect with session
const ws = new WebSocket('ws://localhost:8090/ws?
    session_id=your_session_id&client_id=dashboard');

// Listen for events
ws.onmessage = function(event) {
    const data = JSON.parse(event.data);
```

```
    console.log('Received event:', data);  
};
```

Server to Client Events

```
// Translation progress  
{  
  "type": "translation_progress",  
  "session_id": "session-123",  
  "step": "translation",  
  "message": "Translating line 5/10",  
  "progress": 55.0,  
  "current_item": "line_5",  
  "total_items": 10,  
  "timestamp": 1640995200,  
  "worker_info": {  
    "host": "localhost",  
    "port": 8444,  
    "type": "ssh-llamacpp",  
    "model": "llama-2-7b-chat",  
    "capacity": 10  
  }  
}  
  
// Translation completed  
{  
  "type": "translation_completed",  
  "session_id": "session-123",  
  "message": "Translation completed successfully",  
  "progress": 100.0,  
  "timestamp": 1640995300  
}  
  
// Error event  
{  
  "type": "translation_error",  
  "session_id": "session-123",  
  "step": "translation",  
  "message": "Failed to translate line 5",  
  "error": "Connection timeout",  
  "timestamp": 1640995200  
}
```

HTTP API (for REST endpoints)

```
# Get server status  
curl http://localhost:8090/status  
  
# Get active sessions  
curl http://localhost:8090/sessions
```

```
# Get session history
curl http://localhost:8090/sessions/history
```

Testing

Unit Tests

```
# Test WebSocket server
go test ./cmd/monitor-server
```

```
# Test event system
go test ./pkg/events
```

```
# Test SSH workers
go test ./pkg/sshworker
```

Integration Tests

```
# Test full WebSocket monitoring workflow
go run demo-websocket-client.go
```

```
# Test SSH worker integration
go run demo-ssh-worker-with-monitoring.go
```

```
# Test LLM integration
go run demo-real-llm-with-monitoring.go
```

Load Testing

```
# Run multiple concurrent translation sessions
for i in {1..5}; do
  go run demo-translation-with-monitoring-fixed.go &
done
```

```
# Monitor all sessions in the dashboard
open http://localhost:8090/monitor
```

Troubleshooting

Common Issues

1. WebSocket Connection Failed

Solution: Check if monitoring server is running on port 8090
Command: `lsof -i :8090`

2. SSH Worker Connection Failed

Solution: Verify SSH credentials and connectivity
Command: `ssh milosvasic@localhost 'echo "SSH works"'`

3. LLM API Authentication Failed

Solution: Check API key environment variables
Command: `echo $OPENAI_API_KEY`

4. Port Conflicts

Solution: Change server port in configuration
File: `internal/working/config.distributed.json`

Debug Mode

Enable debug logging:

```
# Set log level
export LOG_LEVEL=debug

# Run monitoring server with debug output
go run ./cmd/monitor-server -log-level=debug
```

Performance Considerations

WebSocket Connection Limits

- Default: 100 concurrent connections
- Recommended: Monitor connection count in production

SSH Worker Scaling

- Maximum workers: 20 (configurable)
- Connection timeout: 30 seconds (configurable)
- Health check interval: 30 seconds

Memory Usage

- Event history: 1000 events per session
- Session timeout: 24 hours
- Log retention: 7 days

Security Considerations

WebSocket Security

- Consider using WSS in production
- Implement authentication tokens

- Rate limiting for connections

SSH Security

- Use SSH key authentication instead of passwords
- Limit command execution permissions
- Regular key rotation

API Security

- Implement API key authentication
- Rate limiting for HTTP endpoints
- Input validation and sanitization

Future Enhancements

Planned Features

- ☐ User authentication and authorization
- ☐ Persistent session storage
- ☐ Advanced analytics and reporting
- ☐ Mobile monitoring app
- ☐ Integration with monitoring systems (Prometheus, Grafana)

Performance Improvements

- ☐ Event batching for high-frequency updates
- ☐ WebSocket compression
- ☐ Connection pooling for SSH workers
- ☐ Caching for translation results

UI/UX Enhancements

- ☐ Real-time notifications
- ☐ Customizable dashboards



Dark mode support



Mobile-responsive design

Contributing

Development Setup

```
# Clone the repository  
git clone <repository-url>
```

```
# Install dependencies  
go mod tidy
```

```
# Run development server  
go run ./cmd/monitor-server
```

```
# Run tests  
go test ./...
```

Code Style

- Follow Go conventions
- Add comprehensive tests
- Update documentation
- Use meaningful commit messages

Support

Getting Help

- Check this documentation
- Review GitHub issues
- Contact the development team

Reporting Issues

- Include error logs
- Provide configuration details
- Describe steps to reproduce
- Include system information